features

“I’m using **classical computer vision**, not ML training. By features, I mean **geometric and pixel-based properties**, such as continuous straight lines for axes, connected pixel clusters for symbols, and centroids of those clusters for data points.”

how does it work

“The pipeline is rule-based: grayscale → thresholding → line detection for axes → contour detection for symbols → centroid calculation → pixel-to-value mapping.”

The goal is to fully understand how an image becomes a mathematical object before introducing ML

What I have done so far is:

- Environment setup (Colab / Python / OpenCV / NumPy / Matplotlib)
- Image loading and inspection
- Understanding image representation (RGB → grayscale)
- Manual thresholding
- Binary image generation
- Edge detection experiments (Canny)
- Initial axis-structure understanding

Features here are:

- Edges
- Line orientation
- Pixel connectivity
- Shape geometry
- Relative position

how

- A long horizontal connected edge → likely x-axis
- A cluster of pixels forming a circle → data point
- Distance between tick marks → scale

These are **geometric features**, not learned features.

Next alg:

Axis refinement

Choose the *longest* horizontal and vertical line

Tick mark detection

Short perpendicular lines near axes

Pixel → coordinate mapping

Convert pixel distance to numerical scale

Point detection

Find connected components (symbols)

Centroid extraction

Get (x, y) per symbol

CSV generation

Final output

deterministic image-processing foundation: loading real scientific plots, converting them to grayscale and binary form, detecting edges, and using Hough transforms to identify and classify axes geometrically. The goal right now is to fully understand and validate plot structure before moving to data extraction or machine learning.

Main Project:

connects your Google Drive to the Colab runtime.

Every time:

- the runtime restarts
- the session times out
- you close the browser

👉 **all local files are erased**

Mounting Drive solves this.

Why we need it

OpenCV is used to **process images**, not just view them.

With cv2 we can:

- read images (cv2.imread)
- convert color spaces (RGB → grayscale)
- apply thresholding
- detect edges
- detect lines (axes)
- work with pixel-level data

Python by itself **cannot do image processing** like this efficiently.

How to explain it simply

You can say:

"OpenCV provides the core image-processing operations needed to convert a plot image into numerical structures like edges, lines, and shapes."

matplotlib is a **plotting and visualization library**.

- showing original images
- showing grayscale images
- showing thresholded images
- showing detected edges and lines

Load image

This code loads the scatter plot image as a numerical array using OpenCV.
The image is converted from BGR to RGB so it displays correctly in Matplotlib.
The image is then displayed to visually verify that it loaded correctly.

The axes are hidden to avoid clutter, and the image shape is printed to confirm its dimensions and color channels before preprocessing.

Rgb to gray

original color image into a grayscale image by removing color information and keeping only intensity values.
Grayscale simplifies the image while preserving all structural features such as axes, curves, and text.
This reduction makes thresholding and edge detection more reliable and computationally efficient.
The printed shape confirms that the image is now a single-channel matrix while maintaining the original spatial resolution.

This image is a grayscale version of a rheological scatter plot showing storage modulus G' and loss modulus G" as a function of angular frequency ω.
The x-axis represents angular frequency in rad/s on a logarithmic scale, and the y-axis represents modulus values in Pascals, also on a logarithmic scale.
The plot contains two data series corresponding to G' and G", shown as discrete scatter points rather than continuous lines.
The dark rectangular border corresponds to the plot frame and axes, while the lighter curved regions correspond to the experimental data points.
Text annotations and arrows indicate characteristic relaxation times, which appear as darker regions due to high contrast in grayscale.

This represents the shape of the grayscale image matrix, where 268 is the image height in pixels and 338 is the image width in pixels.
Each value in this matrix corresponds to the intensity of a single pixel in the image.
Since the image is grayscale, there is no third channel dimension.
This confirms that the spatial structure of the original image is preserved for further processing.

What this step does (thresholding)

In this step, I apply binary thresholding to the grayscale image to separate the plot content from the background.
Each pixel is compared against a chosen threshold value, and based on that comparison it

is converted into either black or white.
This converts the image into a binary matrix that highlights axes, data points, and text while suppressing background noise.

Why three different thresholds (180, 200, 220)

I intentionally tested multiple threshold values to understand how sensitive the image is to threshold selection.
Lower thresholds preserve more information but may introduce noise, while higher thresholds remove noise but risk losing thin features.
By visual comparison, threshold 200 provided the best balance between retaining structural features and suppressing background artifacts.

Why this is done manually (important)

Threshold selection is currently manual because different published figures have different contrast, resolution, and background characteristics.
At this early stage, manual thresholding helps build intuition about how the image behaves before automating parameter selection.
Once we understand this behavior across multiple images, we can later introduce adaptive or automated thresholding.

This is exactly the **right justification** academically.

What the output represents

The output images are binary representations where white pixels correspond to foreground structures such as axes and data points, and black pixels represent background.
This binary format is ideal for downstream structural detection algorithms.

What algorithm comes next and why

Next algorithm: Edge Detection (Canny)

After thresholding, the next step is edge detection using the Canny algorithm.
Canny identifies sharp intensity changes, which correspond to boundaries such as plot

borders, axes, and curve outlines.
This converts the binary image into an edge map that highlights only structural boundaries.

Why Canny is used

Canny is chosen because it is robust to noise, preserves thin edges, and provides precise localization of boundaries.
This makes it well suited for detecting long, straight edges like axes.

What comes after edge detection (preview)

Once edges are detected, I use the Hough Line Transform to identify long straight lines.
Based on geometric orientation, horizontal and vertical lines are separated to identify x- and y-axes.
This step is deterministic and explainable, which is important before introducing any machine learning.

One-sentence summary (if he interrupts)

“I manually explored thresholding to understand image sensitivity, selected a balanced threshold, and then use edge detection followed by Hough line detection to extract axes in a fully explainable way.”

That sentence alone is **excellent**.

This step applies binary thresholding with a selected threshold value of 200 to convert the grayscale image into a binary image.
Pixels darker than the threshold are treated as foreground and set to white, while the rest are treated as background.
I assign this result to a final variable so it can be consistently used in all subsequent processing steps.

We separate the image into foreground and background so the algorithm can focus only on meaningful plot elements.
The foreground contains important structures like axes, curves, and data points, while the background is mostly empty space.
This separation simplifies the image and reduces noise, making later steps like edge detection and line detection more reliable.

Right now, I'm just using Canny edge detection to visualize and verify that the important plot boundaries like axes and curves are being detected correctly.
Next week, I'll use these edges as input to a line-detection algorithm to actually identify the axes and proceed with data extraction.

```
print(type(thresh))

print(thresh.shape)
```

This is just a sanity check to confirm that the thresholded image exists and is in the correct format.
It tells me that thresh is a NumPy array and shows its height and width, which confirms it's ready for the next processing step.

Whats for next week

- **Axis detection using Hough Transform**

Detect and separate horizontal (x-axis) and vertical (y-axis) lines from the edge image.

 Axis filtering and selection

Remove unwanted lines (ticks, labels) and keep only the main plot axes based on length and orientation.

 Validation and visualization

Overlay detected axes on the original image and verify correctness visually